-- 以下为AI生成的图文笔记内容 --

一、*GitHub Packages 00:00*

1. 对包注册表的支持 00:02

对包注册表的支持

语言	说明	包格式	包客户端
JavaScript	节点包管理器	`package.json`	`npm`
Ruby	RubyGems 包管理器	`Gemfile`	`gem`
Java	Apache Maven 项目管理和理解工具	`pom.xml`	`mvn`
Java	Java 的 Gradle 构建自动化工具	`build.gradle` 或 `build.gradle.kts`	`gradle`
.NET	.NET 的 NuGet 包管理	`nupkg`	`dotnet` CLI
空值	Docker 容器管理平台	`Dockerfile`	`Docker`

有关配置包客户端以结合使用 GitHub Packages 的详细信息，请参阅"使用 GitHub Packages 注册表"。

有关 Docker 和 Container registry 的详细信息，请参阅"使用容器注册表"。

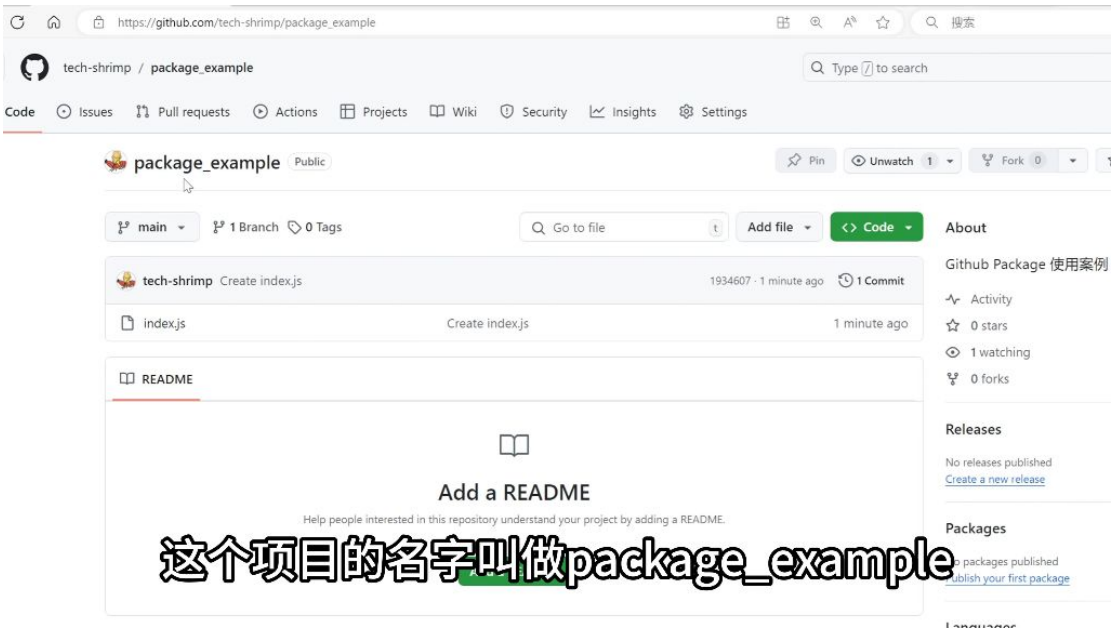
本文内容

关于 GitHub Pac
关于 GitHub Pac
支持的客户端和
向 GitHub Packa
管理包
联系支持部门

- **多语言支持**：支持JavaScript、Ruby、Java、.NET等多种语言的包管理工具
- **包格式说明**：
  - JavaScript使用package.json
  - Ruby使用Gemfile
  - Java使用pom.xml或build.gradle
  - .NET使用nupkg
  - Docker使用Dockerfile
- **功能定位**：GitHub Packages是GitHub提供的包管理仓库功能，可以作为官方仓库的替代方案

2. 应用案例 00:44

1）项目创建与配置



- **基础配置**：
  - 创建名为package_example的项目
  - 包含简单的index.js文件，内容为"hello world"
- **环境准备**：
  - 安装Node.js开发环境
  - 验证环境：执行node -v和npm -v检查版本
- **项目初始化**：
  - 执行npm init初始化项目
  - 包命名规范：使用@用户名/包名格式
  - 测试命令配置为exit 0

2）发布包的配置 02:27

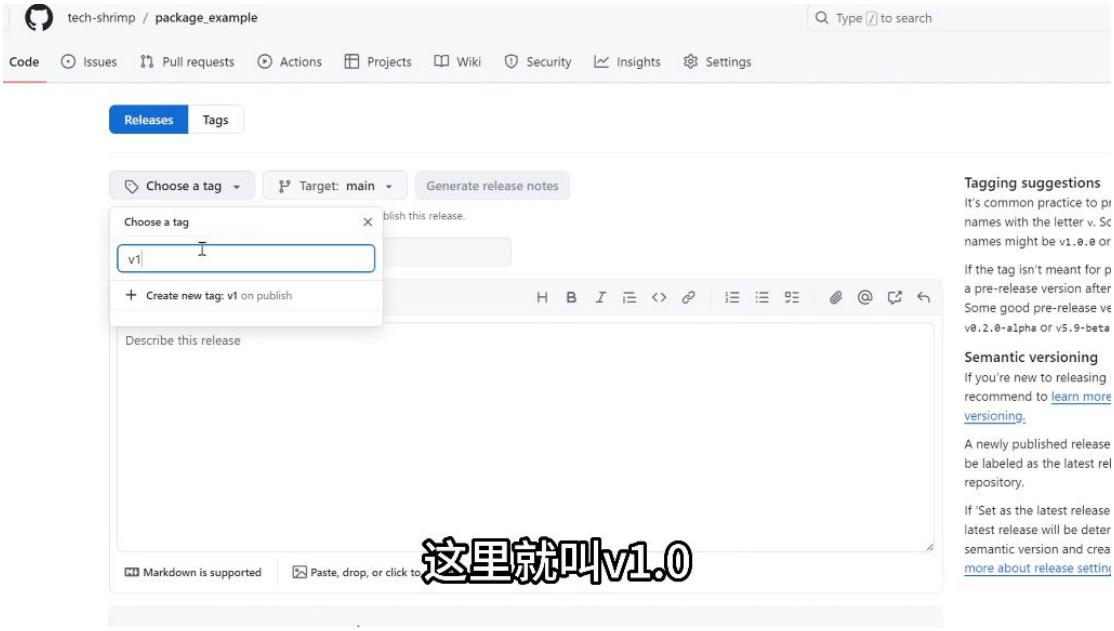
release_package.yml

D: > LearnGit > package_example > .github > workflows > release_package.yml

```yaml
name: Node.js Package

on:
  release:
    types: [created]

jobs:

  publish-gpr:
    runs-on: ubuntu-latest
    permissions:
      packages: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 16
          registry-url: https://npm.pkg.github.com/
      - run: npm ci
      - run: npm publish
        env:
          NODE_AUTH_TOKEN: ${{secrets.GITHUB_TOKEN}}
```

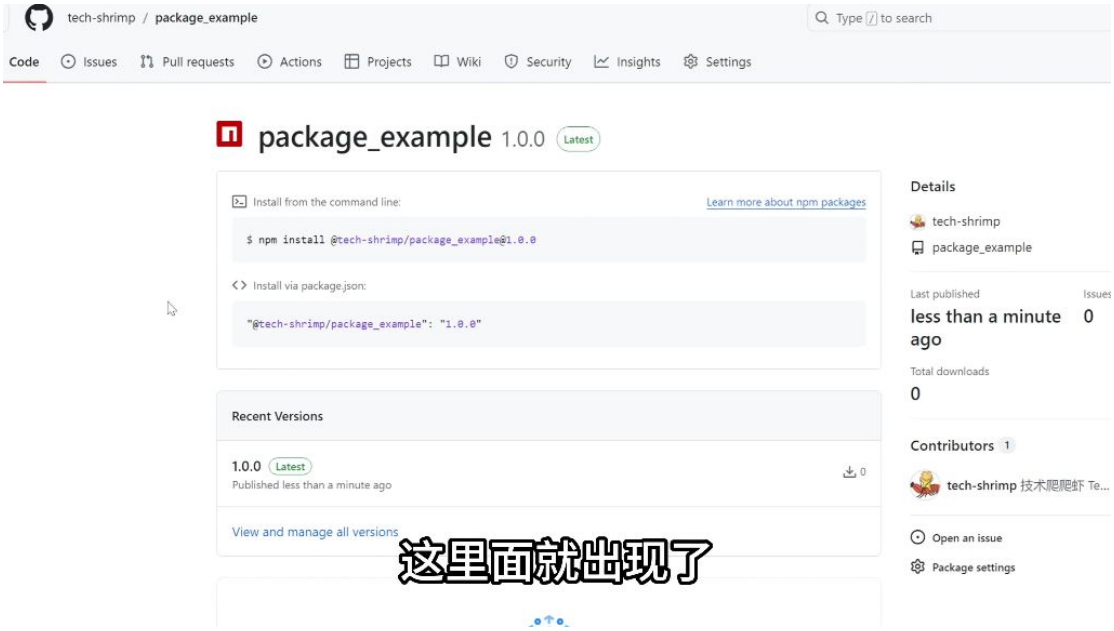
-
- **工作流文件创建**：
  - 在.github/workflows目录下创建release_package.yml
  - 触发器设置为release创建时触发
- **关键配置项**：
  - 使用https://npm.pkg.github.com/作为registry-url
  - 权限设置：packages:write和contents:read
  - 使用${secrets.GITHUB_TOKEN}进行认证
- **发布流程**：
  - 检出代码
  - 设置Node环境
  - 执行npm ci和npm publish

3）创建Release触发Action 03:44



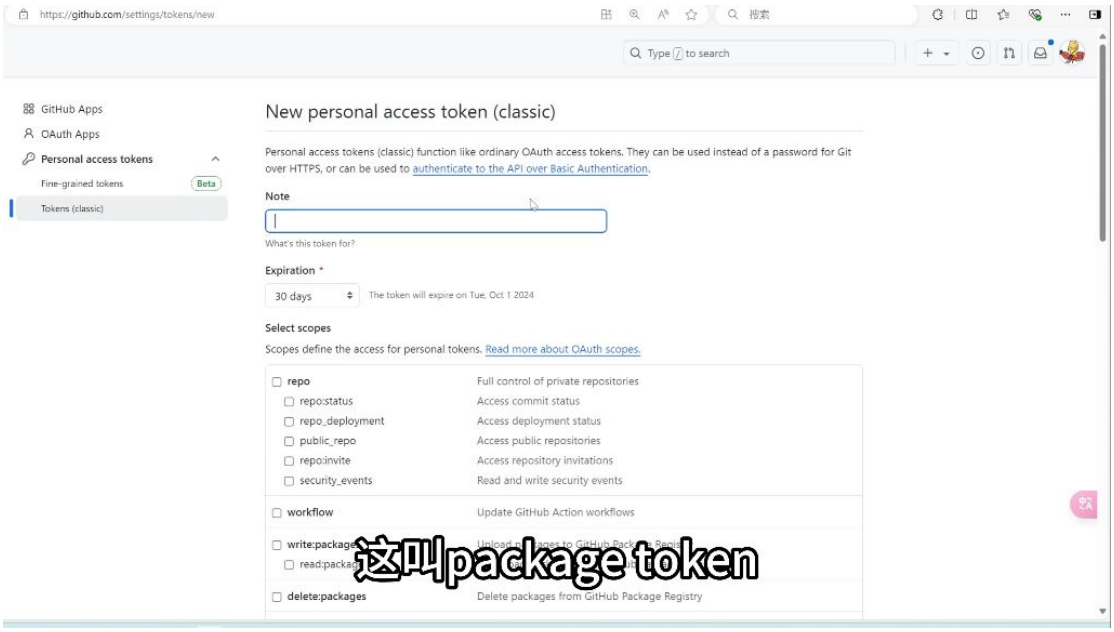
- **Release创建步骤**：
  - 创建v1.0标签
  - 填写版本说明和功能描述
  - 点击publish发布
- **验证发布**：
  - 在Actions标签页查看发布流程执行情况
  - 成功后在Packages标签页查看发布的包

4）使用GitHub Packages发布的包 04:58



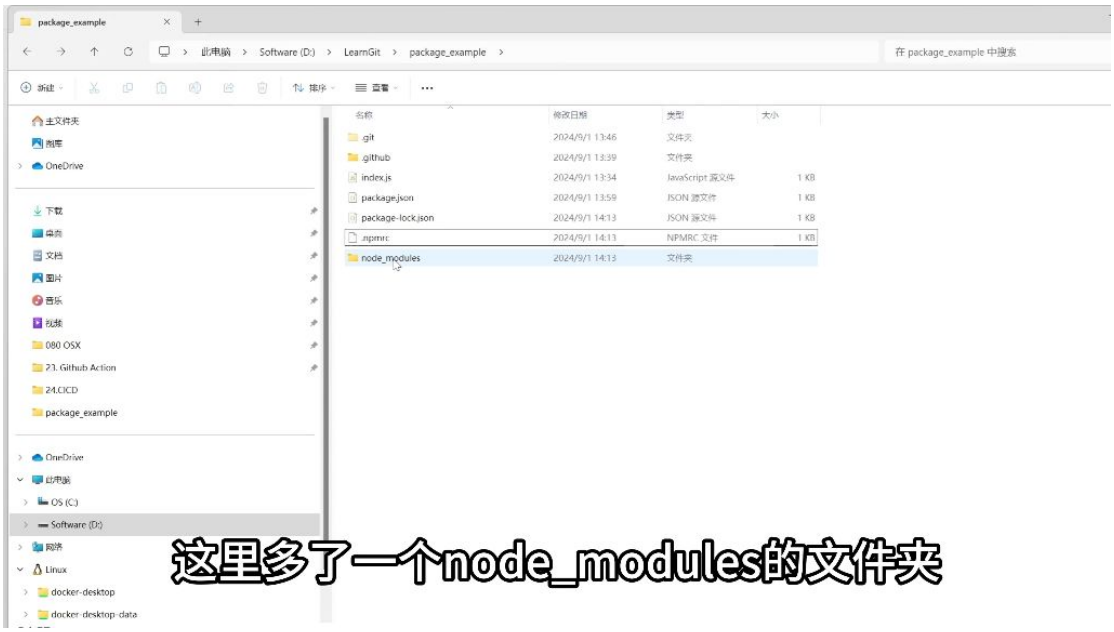
- **配置依赖**：
  - 在package.json的dependencies中添加"@tech-shrimp/package_example":"1.0.0"
- **仓库声明**：
  - 创建.npmrc文件
  - 添加@tech-shrimp:registry=https://npm.pkg.github.com
- **认证配置**：
  - 在.npmrc中添加认证token
  - 格式：//npm.pkg.github.com/:_authToken=token值

5）创建GitHub Token 05:19

- 

- **创建步骤**：
  - 在Settings > Developer settings中创建Personal access token
  - 选择classic类型
- **权限设置**：
  - 勾选所有package相关权限
  - 包括write:packages、read:packages和delete:packages
- **注意事项**：
  - 创建后立即复制token
  - token只显示一次，无法再次查看

6）安装托管在GitHub上面的包 06:04

- 

- **安装流程**：
  - 执行npm install命令
  - 验证node_modules目录生成
- **配置要点**：
  - 确保.npmrc配置正确
  - 包含registry声明和authToken
- **使用总结**：
  - 依赖声明和仓库配置两步即可使用GitHub Packages
  - 适用于私有包管理和团队协作场景

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
GitHub Package 简介	GitHub 提供的包管理仓库功能，支持推送和使用软件包（如 npm、Maven、Docker 等），公开仓库免费，私有仓库提供 **500MB 存储空间**和 **1GB/月传输带宽**	与官方包管理器（如 npm）的区别及适用场景	★★

npm 包初始化	通过 npm init将项目初始化为 npm 包，生成 package.json 文件	package.json中 name 字段格式（需包含仓库地址前缀）	★★
GitHub Action 自动化发布	创建 .github/workflows/release-package.yml文件，定义触发条件（Release 创建时）和发布步骤（检出代码、配置 Node 环境、推送至 GitHub Package）	**关键配置**：registry-url需指向 npm.pkg.github.com	★★★
GitHub Package 使用配置	1. 在 package.json中添加发布地址 2. 创建 .npmrc文件声明 GitHub 为包仓库，并配置 **Personal Access Token**	Token 需包含 package权限，且需替换用户名	★★★★
安装 GitHub Package	通过 npm install安装托管在 GitHub 的包，依赖项需在 dependencies中声明包地址	需确保 .npmrc中 Token 和仓库地址正确	★★★